

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

1-mashg'ulot. Tarmoqda ma'lumot almashuvchi klient-server dasturini tuzish.

O'quv savollari:

1. Socket modulining asosiy metodlari bilan tanishish.
2. TCP protokoli asosida klient-server ulanishini tashkil qilish.
3. Ma'lumotlarni uzatish va qabul qilish jarayonini amalda bajarish.
4. Klient va server dasturlarida xatoliklarni aniqlash va bartaraf etish.

1. Socket modulining asosiy metodlari bilan tanishish

Socket tushunchasi

Socket — bu **tarmoq dasturlashning asosiy komponenti** hisoblanadi. U kompyuterlar yoki dasturlar o'rtasida tarmoq orqali ma'lumot almashishni ta'minlaydigan **aloqa nuqtasi** (inglizcha — *endpoint*) sifatida ishlaydi.

Socket yordamida ikki dastur o'rtasida muloqot (ya'ni, ma'lumot uzatish va qabul qilish jarayoni) o'rnatiladi. Bir dastur odatda **server** rolini bajaradi — ya'ni, u ma'lum portda ulanishlarni “tinglaydi”, ikkinchi dastur esa **klient** sifatida shu serverga ulanadi. Ulanish o'rnatilgandan so'ng ular o'zaro baytlar ko'rinishida xabar almasha oladi.

Ta'rif:

Socket — bu tarmoqda ma'lumot almashishni tashkil etuvchi dasturiy interfeys bo'lib, u dasturlar o'rtasidagi aloqa uchun “eshik” vazifasini bajaradi.

Misol uchun:

Agar siz maktub yuborayotgan bo'lsangiz:

- Siz — **klient** bo'lasiz.
- Qabul qiluvchi — **server**.
- Pochta manzili va uy raqami — bu **IP manzil va port**.
- Konvertni tashlab yuboradigan joy — bu **socket**.

Ya'ni, socket orqali siz xabarni tarmoqqa “jo'natasiz”, server tomondagi socket esa uni “qabul qiladi”.

Shuning uchun:

- Socket **dasturiy komponent** hisoblanadi; uni Python, C++, Java kabi tillarda kod orqali yaratasisiz.
- U **kompyuterning tarmoq adapteri (NIC)** bilan hamkorlikda ishlaydi, lekin o'zi temir qurilma emas.
- Har bir socket **IP manzil + port raqami** orqali taniladi.
- **IP manzil** — tarmoqdagi kompyuterning manzili.
- **Port raqami** — aynan shu kompyuterda qaysi dastur yoki xizmat bilan aloqa o'rnatilayotganini bildiradi.

Masalan, 192.168.1.10:8080 degan yozuvda:

- 192.168.1.10 — bu kompyuterning IP manzili,

- 8080 — bu shu kompyuterdagi tarmoq xizmatining port raqami.

Shunday qilib, har bir socket manzili (IP + port) orqali tarmoqdagi boshqa socket bilan o'zaro aloqa o'rnatishi mumkin.

Eng muhimi, **socket to'g'ridan-to'g'ri transport qatlamidagi protokollarga (masalan, TCP yoki UDP)** bog'liq bo'lmaydi. Dasturchi qaysi turdagi transport protokolidan foydalanishni o'zi tanlaydi:

- Agar **TCP** tanlansa, socket ishonchli, oqimli (stream-based) muloqotni ta'minlaydi.
- Agar **UDP** tanlansa, socket tez, lekin ishonchsiz (datagram-based) aloqa turida ishlaydi.

Demak, socket — bu **transport protokollarini bir xil interfeysda boshqarish imkonini beruvchi abstrakt aloqa nuqtasi**.

U orqali:

- server va klient o'rtasida ulanish o'rnatiladi,
- ma'lumotlar baytlar ko'rinishida uzatiladi,
- xabarlar qabul qilinadi va qayta ishlanadi.

Socketning tarixiy rivojlanishi

Socket tushunchasi birinchi marta 1983-yilda Berkeley Unix tizimida joriy etilgan. Hozirgi kunda socket API de-fakto standart sifatida barcha zamonaviy operatsion tizimlarda qo'llaniladi.

Socket turlari va ularning farqlari

Python socket moduli ikki asosiy socket turini qo'llab-quvvatlaydi:

1. TCP Socketlar (SOCK_STREAM)

- **Ishonchli ulanish:** Ma'lumotlar yo'qolmasligi kafolatlanadi
- **Ketma-ketlik:** Ma'lumotlar yuborilgan tartibda qabul qilinadi
- **Oqim oriyentirlangan:** Ma'lumotlar uzluksiz oqim sifatida uzatiladi
- **Ikki tomonlama muloqot:** Ham yuborish, ham qabul qilish mumkin

2. UDP Socketlar (SOCK_DGRAM)

- **Ishonchsiz ulanish:** Ma'lumotlar yo'qolishi mumkin
- **Paket oriyentirlangan:** Har bir ma'lumot alohida paket sifatida uzatiladi
- **Tezkor:** TCP ga nisbatan kamroq overhead
- **Bir tomonlama muloqot:** Asosan yuborish yo'nalishi ustun

Socket metodlarining klassifikatsiyasi

Ulanish va konfiguratsiya metodlari:

- `socket()` - yangi socket obyektini yaratadi
- `setsockopt()` - socket parametrlarini sozlaydi
- `getsockopt()` - socket parametrlarini o'qiydi

Server tomondagi metodlar:

- `bind()` - socketni lokal manzilga bog'laydi
- `listen()` - kiruvchi ulanishlarni kutish rejimi
- `accept()` - klient ulanishini qabul qiladi

Klient tomondagi metodlar:

- `connect()` - masofaviy serverga ulanadi
- `connect_ex()` - xatolarni batafsilroq ko'rsatadigan ulanish

Ma'lumot almashish metodlari:

- `send()` - ma'lumot yuboradi
- `recv()` - ma'lumot qabul qiladi
- `sendall()` - barcha ma'lumotlarni yuboradi
- `sendto()` - UDP uchun ma'lumot yuboradi
- `recvfrom()` - UDP uchun ma'lumot qabul qiladi

Yopish metodlari:

- `close()` - socketni yopadi
- `shutdown()` - ulanishning ma'lum yo'nalishini yopadi

2. TCP protokoli asosida klient-server ulanishini tashkil qilish

TCP protokoli arxitekturası

TCP (Transmission Control Protocol) - bu transport qatlamidagi ishonchli, ulanishga asoslangan protokol bo'lib, OSI modelining 4-qatlamida joylashgan.

TCP ning asosiy xususiyatlari

1. Ishonchli uzatish

- **Akkumulativ ACK:** Qabul qiluvchi har bir paket uchun tasdiq yuboradi
- **Qayta uzatish:** Yo'qolgan paketlar avtomatik ravishda qayta yuboriladi
- **Cheksum:** Ma'lumotlarning buzilishini aniqlash

2. Oqimni boshqarish

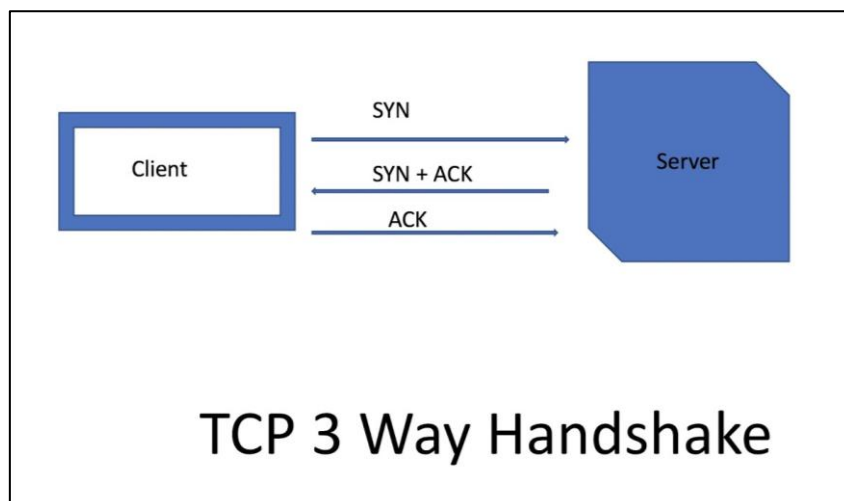
- **Sliding Window:** Yuboruvchi va qabul qiluvchi o'rtasidagi oqimni optimallashtirish
- **Congestion Control:** Tarmoq tiqilishlarini oldini olish

3. Ulanishni boshqarish

- **Three-Way Handshake:** Ulanish o'rnatish jarayoni
- **Graceful Close:** Ulanishni to'g'ri yopish

TCP ulanishi bosqichlari

1. Aloqa o'rnatish (Three-Way Handshake)



SYN (Synchronize): Klient ulanish so'raydi

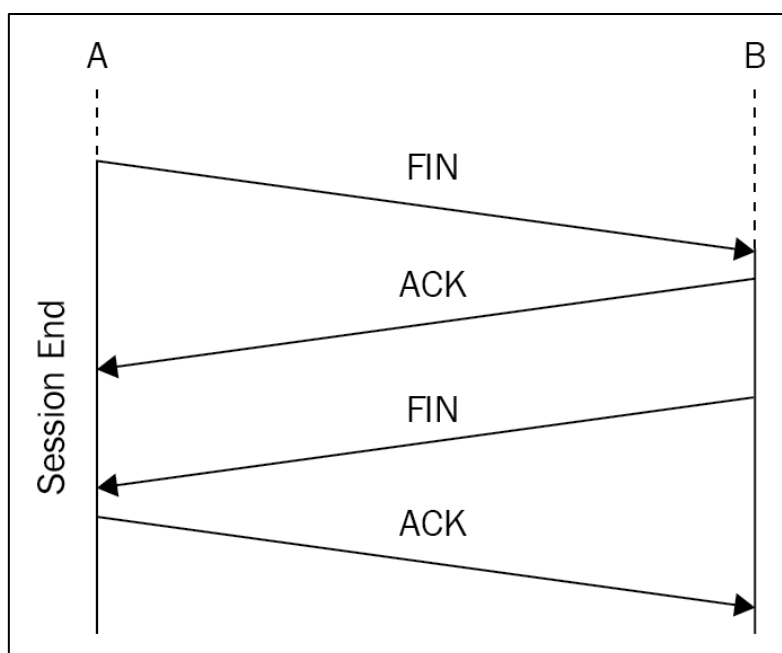
SYN-ACK (Synchronize-Acknowledge): Server javob beradi

ACK (Acknowledge): Klient tasdiqlaydi

2. Ma'lumot almashinuvi

- Ma'lumotlar segmentlarga bo'linadi
- Har bir segment ketma-ket raqamga ega
- Qabul qiluvchi ACK yuboradi

3. Ulanish yopilishi



Server arxitektura modellari

1. Iterativ Server

- Bir vaqtning o'zida faqat bitta klient bilan ishlaydi
- Sodda va oson amalga oshiriladi
- Past unumdorlik

2. Konkurent Server

- Bir nechta klientlarni bir vaqtda qabul qiladi
- Thread yoki process lar orqali amalga oshiriladi
- Yuqori unumdorlik

Port manzil tushunchasi

Port - bu mantiqiy ulanish nuqtasi bo'lib, 0 dan 65535 gacha bo'lgan raqam bilan ifodalanadi.

Port turlari:

- 0-1023: Tizim portlari (well-known ports)
- 1024-49151: Ro'yxatdan o'tgan portlar
- 49152-65535: Dinamik portlar

3. Ma'lumotlarni uzatish va qabul qilish jarayonini amalda bajarish

Ma'lumot uzatishning fundamental tamoyillari

1. Byte oriyentirlangan uzatish

Socketlar byte ketma-ketligi bilan ishlaydi. Barcha ma'lumotlar avval byte formatiga o'tkazilishi kerak.

2. Buffer mexanizmi

Socketlar ichki bufferlarga ega bo'lib, ular:

- Yuborish va qabul qilish operatsiyalarini buffered qiladi
- Tarmoq tezligi va dastur tezligi o'rtasidagi farqni muvozanatlash

Ma'lumot uzatish protokollari

1. Oddiy matn uzatish

- Ma'lumotlar to'g'ridan-to'g'ri yuboriladi
- Hech qanday formatlash talab qilinmaydi
- Oddiy vazifalar uchun mos

2. Formatalangan uzatish

- Ma'lumotlar ma'lum strukturada uzatiladi
- JSON, XML yoki maxsus formatlar qo'llaniladi
- Murakkab ma'lumotlar uchun mos

3. Binar uzatish

- Fayllar va binar ma'lumotlar uzatiladi
- Maxsus kodlash talab qilmaydi
- Samaradorlik yuqori

Xabarlarini ajratish muammosi

TCP oqim oriyentirlangan protokol bo'lgani uchun, xabarlarini qanday ajratish muhim masala hisoblanadi.

Yechim usullari:

1. Belgilar orqali ajratish

- Maxsus ajratuvchi belgi (masalan, yangi qator '\n')

- Sodda va samarali

2. Uzunlik prefiksi

- Har bir xabar oldiga uzunlik ma'lumoti qo'shiladi
- Aniq va ishonchli

3. Chegaralar orqali

- HTTP dagi kabi chegaralar qo'llaniladi
- Murakkab ammo kuchli

Buffer boshqaruvi

Buffer hajmini tanlash

- **Kichik buffer** (256-512 bayt): Tez-tez chaqiruvlar
- **O'rta buffer** (1024-4096 bayt): Umumiy maqsadlar
- **Katta buffer** (8192+ bayt): Fayl uzatish

4. Klient va server dasturlarida xatoliklarni aniqlash va bartaraf etish

Socket dasturlashdagi xato turlari

1. Ulanish xatolari

- **ConnectionRefusedError**: Server mavjud emas yoki rad etdi
- **ConnectionResetError**: Ulanish kutilmaganda uzildi
- **TimeoutError**: Ulanish vaqti tugadi

2. Ma'lumot uzatish xatolari

- **BrokenPipeError**: Ulanish yopilgan, ma'lumot yuborib bo'lmadi
- **ConnectionAbortedError**: Ulanish bekor qilindi

3. Tizim xatolari

- **OSError**: Tizim darajasidagi xatolar
- **PermissionError**: Portga bog'lash uchun ruxsat yo'q